

The Polyglot's Shorthand

An 18.9M-Parameter Encoder for Latinised Code-Mixed Text

Inter IIT Bootcamp 2026 · CS/AI Practice Problem Statement 4

Summary

Consumer text across South Asia arrives Latinised and code-mixed — *bhai order cancel krdo please, urgent meeting h*. Native-script Indic encoders fragment it, English encoders read the syntax as noise, and frontier models handle it well but too slowly to sit at the front of a routing queue.

We built a small encoder for this text specifically. A deterministic phonetic normalisation layer feeds a byte-level BPE vocabulary trained over canonicalised forms, and the input embedding carries four channels: subword identity, position, a hashed consonant skeleton, and six surface features that canonicalisation discards. The skeleton channel is the core idea — *krdo* and *kardo* tokenise to *kr|do* and *kar|do*, sharing no subword, but both reduce to the skeleton *krd* and so share an embedding. Training signal pools across spellings that the subword channel keeps apart.

Parameters	18,868,240 — 3.77% of the 500M ceiling
Sentiment weighted F1 (SentiMix Hinglish)	0.872
Language ID weighted F1 (HingLID, token-level)	0.970
F1 retained under heavy spelling perturbation	88.5%
F1 cost of removing emoji (n = 232)	−6.9 points
Latency, batch 1, end-to-end	3.02 ms
Peak throughput	12,742 sentences/s

Trained from scratch on 1.81M tokens in 25 minutes on a single T4.

1 · Problem and approach

Three established approaches each fail on this traffic. Native-script encoders like IndicBERT and MuRIL carry Devanagari vocabularies, so a Romanised Hindi word has no merge path and shatters into character fragments; transliterating back first is lossy in exactly the ambiguous cases, since *kr* could be *kar*, *kir* or *kur* and the transliterator must guess before the encoder sees anything. English encoders handle embedded English words fine but find the Hindi matrix syntax holding them together unmotivated. Frontier models fail economically rather than linguistically — a router classifying every message in single-digit milliseconds cannot call one.

The problem statement names three properties to absorb, and each maps to a mechanism here: phonetic volatility (canonicalisation plus the skeleton channel), intra-sentence mixing (one shared vocabulary, with token-level language identification as an auxiliary task), and pragmatic inversion by emoji (atomic emoji tokens plus the surface-feature channel). The last is easiest to lose: a pipeline that strips emoji as noise deletes the token carrying the sentiment, and 17.4% of our training tweets contain at least one.

The thesis is that the gap between *parsing this text* and *deploying at the front of a queue* is closed by the input representation, not by a smaller general model. If the representation absorbs spelling variation before the transformer sees it, the transformer can be small.

Data. SemEval-2020 Task 9 (SentiMix Hinglish) for sentiment — 15,130 train rows, 3,000 labelled validation, 3,000 unlabelled — and L3Cube-HingLID for word-level Hindi/English tags, 31,756 / 6,279 / 6,420 sentences. SentiMix carries its own word-level language tags, which we fold into the identification task, adding 261,284 labelled tokens inside the sentiment domain. Pretraining uses every sentence from both: 65,585 sentences, 1,813,240 tokens. One data note: the unlabelled split was written UTF-8-as-cp1252, so every emoji arrived corrupted; round-tripping through cp1252 repaired 2,431 of 2,642 damaged tokens.

2 · Design

Tokenisation. The pipeline is word-centric — each word becomes three representations independently, and the subword sequence is assembled afterwards, since a word-level property cannot be attached to a stream that has forgotten where words began. Canonicalisation lowercases, caps character runs at two, applies 18 digraph folds (aa→a, ee→i, ph→f, sh→s, ai→e) and a single-character map (z→j, w→v, q→k, y→i). The vocabulary is byte-level BPE, 16,000 merges plus 1,291 emoji and placeholder entries added atomically, totalling 17,291. Byte-level matters because there is no out-of-vocabulary path at all: an unfamiliar spelling costs extra subwords, never lost meaning. Measured fertility is **1.369 subwords per word** clean and **1.477** after heavy perturbation — a 7.9% increase, meaning severe misspelling costs less than one extra subword per twelve words. Across the corpus, 108,637 surface words reduce to 72,871 canonical forms and 27,182 skeletons: a third of the apparent vocabulary is spelling variation, and by the aggressive measure three-quarters is.

The skeleton channel. A word's canonical form reduced to first character plus non-initial consonants, truncated at eight, hashed into 8,192 buckets. kar, kr, karr and kaar all reach kr; nahi, nhi and nahee reach nh; cancell and cancell reach cnc l. A skeleton under two characters falls back to the canonical form, so ki, ka and ko stay distinct. Collisions run about 3.3 skeletons per bucket, tolerable because the subword channel disambiguates.

The surface-feature channel restores six values canonicalisation destroys: is-emoji, is-punctuation, contains-digit, elongation, all-caps, and length. Elongation and capitalisation are the intensity markers this register uses — *bohoooot* is not *bohot*.

Encoder. Six pre-LN blocks, model width 384, six heads, feed-forward 1,152. Pre-LN trains stably from random initialisation without a tuned warmup, which matters with no checkpoint to fall back on. Attention uses PyTorch's fused scaled-dot-product kernel. Four heads share the encoder: a masked-language head tied to the input embedding, sentiment over a pooled summary position, a per-token language tagger, and a serving graph of encoder plus sentiment head.

Component	Parameters
Token embedding (17,291 × 384)	6,639,744
Skeleton embedding (8,192 × 384)	3,145,728
Six transformer blocks	8,875,008
Position, features, norms, pooler, heads	207,760
Total	18,868,240

More than half the parameters are lookup tables rather than compute — a fact §6 returns to.

Training. Masked-language pretraining, 15% of positions, 14 epochs in 19.7 minutes; loss falls 7.41 → 3.12 and is still improving at the end. The skeleton is zeroed at masked positions, without which the objective is nearly degenerate. Fine-tuning interleaves both tasks for six epochs in 4.9 minutes. 30% of training rows are re-spelled each epoch using the perturbation operators — real sentences, real labels, only the spelling moves.

3 · Results

Task	Accuracy	Macro F1	Weighted F1
Sentiment (n = 1,500)	0.8720	0.8735	0.8718
Language ID (n = 193,540 tokens)	0.9697	0.9633	0.9696

On the 0.750 leaderboard figure. The best SemEval-2020 Task 9 system reported 0.750 weighted F1. Ours is 0.872, and the comparison is not valid. Gold test labels were never released, so we evaluate on half the labelled validation file, which is drawn closer to the training distribution; the pretraining stage consumed that text unlabelled, making the encoder transductive with respect to its own test set; and 1,500 rows is a fifth of the official evaluation size. The defensible claim is narrower — an 18.9M-parameter encoder trained from scratch on 1.8M tokens is competitive in-domain with models two orders of magnitude larger.

Robustness. The same test sentences re-spelled at four strengths, labels invariant by construction.

Perturbation	Sentiment	Language ID	Retention
--------------	-----------	-------------	-----------

0 (control)	0.8718	0.9696	100%
1	0.8184	0.9494	93.9%
2	0.7758	0.9370	89.0%
3	0.7718	0.9246	88.5%

The curve flattens — levels 2 and 3 differ by 0.4 points against 5.3 for level 1. Once spelling drifts past what canonicalisation folds, the skeleton channel carries the word and further corruption changes little.

Emoji. On the 232 test rows carrying an emoji, stripping them drops F1 from **0.8783 to 0.8095**, a 6.9-point fall, while rows without emoji sit at 0.8704 and the full set moves only 1.1 points. The model is genuinely using the pragmatic channel.

4 · Error analysis

Slice	n	Weighted F1
Mostly Hindi (English < 20%)	554	0.8628
Balanced (20–60%)	622	0.8633
Mostly English (> 60%)	324	0.8998
Contains emoji	232	0.8783
Contains elongation	282	0.8931
All	1,500	0.8718

The code-mix slices are the diagnostic that matters, and they come out well. A 3.7-point edge on English-dominant rows is small enough to rule out the failure this design exists to avoid — the model is not merely reading the embedded English lexicon. Performance is also flat across length and fertility, so fragmentation is not costing accuracy at inference.

Of 192 errors, neutral absorbs the most and direct positive-negative confusion is rare (27 of 922 polar rows). The eight most confident mistakes show one dominant pattern: five are gold-neutral predicted positive on congratulatory or festival text — *Happy Birthday*, *EidMubarak*, *Thank you for such an awesome interview*. The model reads surface affect; annotators recorded speaker stance, treating formulaic well-wishing as neutral. Roughly a third of the confident error budget is annotation convention rather than model failure. The remainder are genuine pragmatic inversions — polite surface with hostile intent — which is exactly where a larger model would do better.

For language identification, accuracy on words never seen in training is **0.777** against a 0.707 majority baseline. With no memorised label available the model works from subword shape, skeleton bucket and context; the margin is real but modest, and it is the number we would put under the most scrutiny.

5 · Latency and throughput

Device / precision	Batch	p50 (ms)	p99	Throughput
GPU fp32	1	2.997	4.092	334
GPU fp32	8	3.198	4.147	2,502
GPU fp32	32	10.267	10.669	3,117
GPU fp16	128	10.046	10.296	12,742
CPU int8	1	9.617	13.074	104

The knee sits between batch 8 and 32: going from 1 to 8 costs 0.2 ms and buys 7.5× throughput, essentially free because the model is launch-overhead bound. Past 32, fp32 throughput saturates near 3,100 sentences/s and latency scales linearly, so batching further only adds queueing delay. A production router should sit at batch 8–16. Half precision is identical at batch 1 but gives 4.2× throughput at batch 128 once tensor cores saturate. Sequence

length barely matters — 2.8 to 3.1 ms across lengths 16 to 96 — so the full window is free.

Int8 dynamic quantisation on CPU costs nothing measurable in accuracy ($0.8718 \rightarrow 0.8724$) for a $1.29\times$ speedup, making commodity CPU a viable deployment. Preprocessing measures 0.0228 ms per sentence, 0.76% of the forward pass; we had expected it to be a meaningful share at this model size and it is not. End-to-end latency at batch 1 is **3.02 ms**.

6 · Limitations

The headline number is not a leaderboard number, for the three reasons in §3. Pretraining is 1.81M tokens, three orders of magnitude below HingBERT, and loss was still falling at the final epoch. Everything is a single seed, so differences of a point or two between slices are not distinguishable from noise — the 6.9-point emoji delta survives this, the 3.7-point slice gap does not. Drift augmentation and the skeleton channel are both active, so the robustness curve cannot be attributed to the architecture until the ablation runs. And we argue against IndicBERT and mBERT from first principles and measured fertility without having fine-tuned them on this split.

The gap that matters most. SentiMix is political Twitter; the problem statement describes support queues. Our qualitative probes — *bohota badhiya service thi* with a positive or sarcastic emoji, *item delivered bol rha h but mila hi nhi* — all return neutral at high confidence, including cases where the emoji should flip the label. The model scores 0.872 in-domain and transfers poorly to the register actually described. This appears nowhere in the aggregate metrics and is the finding we would prioritise.

7 · Where the architecture should go next

The parameter budget is $26\times$ larger than what we used, so almost every idea below is affordable.

Rebalance parameters away from lookup tables. Embeddings are 52% of the model while doing far less work than the blocks. Factorising the token embedding through a low-rank bottleneck, as ALBERT does, would free roughly 4M parameters to spend on depth — plausibly the single highest-return change at fixed size.

Learn the channel fusion instead of summing it. The four input channels are currently added with equal weight. A learned gate per channel, conditioned on the token, would let the model lean on the skeleton exactly when the subword path is unreliable — badly misspelled or unseen words — and ignore it when the surface form is already clean. The drift curve's flattening after level 1 suggests the skeleton is doing work only in a sub-population, which is precisely the case a gate exploits.

Train spelling invariance explicitly. Nothing in the current objective asks for drift variants to have similar representations; it emerges as a side effect of shared buckets. An auxiliary contrastive loss pulling a word and its perturbed form together in embedding space would target the paper's central claim directly, and it needs no extra labelled data — the perturbation operators already generate the pairs.

Reduce skeleton collisions. At 3.3 skeletons per bucket, unrelated words share vectors. Raising to 32,768 buckets costs 9M parameters, still under 6% of ceiling; multi-hash Bloom embeddings would get most of the benefit far more cheaply by making collisions differ across hash functions rather than compound.

Replace absolute positions. Learned absolute embeddings cap input at the trained length. Rotary or ALiBi positions extrapolate beyond it, which matters for long reviews, and cost nothing at this scale.

Add a character-level channel. A small character CNN over each word would give the model a third view that degrades gracefully where both the subword and skeleton paths fail. The 0.777 unseen-word figure is the specific number this should move.

Scale, and distil. Width 768 with 12 layers remains under 2% of the ceiling. The binding constraint is data, not parameters: L3Cube-HingCorpus offers 1.04B tokens against our 1.8M. Beyond that, distilling a frontier model's judgements onto unlabelled Romanised text transfers its understanding once, offline, into something that can sit in front of a queue — which is the deployment argument this whole design rests on. For the generative tasks the problem statement mentions, a two-to-four layer decoder over this encoder is the right move rather than a separate model.

References

1. Patwa, P. et al. **SemEval-2020 Task 9: Sentiment Analysis of Code-Mixed Tweets**. ACL Anthology 2020. semeval-1.100.
2. Nayak, R., Joshi, R. **L3Cube-HingCorpus and HingBERT**. arXiv:2204.08398, 2022.
3. Lan, Z. et al. **ALBERT: A Lite BERT for Self-supervised Learning**. ICLR 2020.
4. Su, J. et al. **RoFormer: Enhanced Transformer with Rotary Position Embedding**. 2021.
5. Liu, Y. et al. **RoBERTa: A Robustly Optimized BERT Pretraining Approach**. arXiv:1907.11692, 2019.
6. Dao, T. et al. **FlashAttention**. NeurIPS 2022.

Reproducing this. One Kaggle notebook on a T4 with internet enabled, about 30 minutes end to end. Seed 42; width 384, 6 layers, 6 heads; 16,000 BPE merges plus 1,291 emoji and placeholder tokens; 8,192 skeleton buckets; maximum length 96 subwords; pretraining 14 epochs at batch 256 and learning rate 3e-4; fine-tuning 6 epochs at batch 64 and 1e-4; drift augmentation on 30% of rows. The run exports weights, tokenizer, configuration, full metrics, figures and a standalone inference script.